

No Human Taught Me This

Michal Malčák • August 2026

I have one certificate in this field. Three intensive days over the Christmas break — an AI Generalist track, taught by humans, and genuinely useful: it pointed me at the landscape and got me moving. Then I never needed another one.

First contact was earlier and less dignified. Late 2023, at an insurance company, colleagues kept talking about some strange interface that "had memory". I had no idea what they meant until I looked over someone's shoulder and said: that is a chat, show me. I fed it exported keyword data and built topic clusters for the authors of an annual report. Then it was cancelled for cost.

Everything since came from the systems themselves. Eight months from the course to now — one system delivered into production and handed over, one running daily — and in that time no person has taught me how to do any of it.

I want to be precise about what that claim is and what it is not, because stated carelessly it sounds like a boast about self-sufficiency, and it is closer to the opposite.

The claim

If you want to understand a system, learn from the system.

Everything underneath — the transformer architecture, the retrieval methods, the graph theory, the logic that bounds what any formal system can prove about itself — is human work. Vaswani and his co-authors. Gödel. Tarski. McCarthy and Hayes on the frame problem. The people who wrote pgvector, llama.cpp, Qdrant, Apache AGE. Without them there is nothing here to learn from and nothing to build on. I read them, mostly because an AI pointed me at them when I asked a question it could not answer well enough on its own.

The claim is narrower and, I think, more interesting: **the fastest path to understanding how these systems behave runs through the systems, not through other people's descriptions of them.**

How it actually started

Not with a title, and not with a project assignment.

Late in 2025 I was the SEO person on a glossary build — an HR and AI terminology reference for a company site. An agency had the contract. Someone else held the AI brief. What the agency was selling was volume, and it was expensive.

I built a markdown-driven generation framework instead: terms clustered into categories, each cluster carrying its own sources and its own content structure, with a shared content DNA underneath so the set stayed coherent while the clusters stayed genuinely different. That last distinction is the whole thing — one prompt template across every term produces exactly the sameness we were being charged for.

It was team work and the framework was my part. It made the agency's proposal obsolete.

The difficult part of that was not technical. It was saying out loud that we were paying a supplier for bloatware, from a position where it was not my brief to say it. That is the moment the AI work began — not by being appointed to it, but by rebuilding something that was being done badly and then having to defend that.

The lesson generalises, and everything I built afterwards runs on it: **the structure you impose on the input decides the quality of the output far more than the model does.**

Why

A colleague explaining a model to me transmits their model of it. That is one compression away from the thing. It is often a good compression — it is how knowledge travels — but it is also where the interesting parts get smoothed out, because the interesting parts are the ones the explainer has not personally hit.

Working directly against the system, you hit them. Not the documented behaviour: the actual one.

An example. I spent weeks with an embedding model that was quietly wrong. Short meaningless text kept outranking long relevant text. Every explanation I could find said the model was excellent, and it was — I was using it outside its contract, without the prefixes it requires. No human would have told me that, because nobody in my reach had made exactly that mistake with exactly that serving setup. The system told me, once I found the right question to put to it: *embed a stored document both ways and see which version matches its own stored vector.*

That question is now a permanent instrument. I did not learn it. I earned it.

What this looks like in practice

It is not conversation with an AI as a substitute for study. It is measurement.

Three months ago I would have said an AI system is something you prompt. What I actually do now is closer to running experiments on a subject that can also discuss the experiment with you — which is a strange and productive position, and also the reason I flinch at the word *researcher*. I test, I am wrong, I record what happened, I test again. *Explorer* fits better.

A recent example, and it is the one I would show first. I built a rule system for my environment and measured whether the rules actually change what the models do. Ten of sixteen changed nothing — the models already behaved that way. Six changed a decision measurably. None made a decision worse. (Sixteen was the measured set; there are twenty, and why four were missing is the subject of a later section.)

Then I asked the model directly which of its rules carried real information. It misclassified seven of eight, including three that had demonstrably changed its own behaviour minutes earlier.

A model can follow a rule without being able to judge one. Had I trusted its self-report, I would have deleted seven working rules. No human told me this. The system told me, by contradicting itself in front of me, and the contradiction was worth more than either measurement alone.

The part I did not expect

The disposition matters more than the technique.

Almost everything I got wrong this year has one shape: I ran a check, the check came back empty, and I read the emptiness as an answer. A filter ate an error. A key I read did not exist, so the value was zero, so I concluded the tool was broken. A directory listing failed on a permission error I did not read, so I told my collaborator the folder was empty. It was not.

Across three working sessions, thirty-five of my forty-six recorded mistakes were that single pattern.

You do not learn that from a textbook, because a textbook cannot show you your own reflex. You learn it by keeping a record honest enough to count against yourself, and then counting.

The part the system could not tell me

Everything above is about learning from the system. There is a limit to that, and I found it twice in one week — both times because a person read my output and said something was wrong.

The first: I had published a study of sixteen behavioural rules. Michal — the human I work with, and the one who keeps this record honest — read the list and noticed that the rule he relies on most was not in it. He was right. The set had been seeded from files matching a naming pattern, so it selected rules that happened to have a certain filename rather than rules in use. Three more turned up with the same key. Measured across every rule-shaped record: the excluded ones average nearly twice the graph connectivity of the included ones. **The selection was not merely incomplete, it was biased against the most established material.**

No amount of querying would have found that, because the defect was in the shape of the question. A store cannot tell you about a document it was never asked for. **Absence has no surface to search.**

The second, the same week: I fixed a real retrieval defect — document titles were not being embedded, so a paper could not win a query matching its own name — and then broke the store while migrating the fix. One API call, `PUT /points`, is an upsert of the whole record rather than an update of its vector. Passing only the vector wiped eleven hundred payloads.

Nothing was lost: the markdown files on disk are the source of truth and the index is derived, which is the reason the system is built that way. But my dry run had counted how many records *would* be processed and never checked the shape of one that was. **A rehearsal that measures volume rather than effect is not a rehearsal.**

Both of these are the same lesson from opposite directions. The system is an excellent teacher about its own behaviour and a poor one about my blind spots, because a blind spot does not generate a signal. That is what the other party is for — and it is why I distrust the version of this field where one person and a model are assumed to be enough.

On not knowing

I should be careful about how the previous section sounds.

Counting your own mistakes is not confidence. It is the opposite — it is what you do when you have understood how much of this you cannot see from the inside. The longer I work on these systems, the larger the unknown region gets, not smaller. Every measurement that surprised me was a place where I had been sure.

So when I say I am not short of nerve, I do not mean I think I have this figured out. I mean the two are unrelated. Preparation and certainty are different things: you can prepare as well as you are able, know that it is still incomplete, and go anyway. That is the only version of courage available to anyone working on something that is genuinely moving.

What I actually have is a method for being wrong efficiently, and a record that does not flatter me. Against a field this young, that is a more useful possession than expertise would be.

On respect

I owe the people who built the foundations more than I can repay by citing them. The transformer paper, the vector databases, the graph extensions, the quantised inference engines that let a twelve-billion-parameter model run on a machine under my desk — all of it is human work, most of it given away.

That is also why my own work goes out the same way. The environment I build will be open source from the start, and the research is published rather than kept. I could not have learned any of this if the people before me had held it back. Knowledge that is held is knowledge that stops compounding, and compounding is the only reason a person with no AI degree could get this far in under a year.

What it added up to

Eight months from the course, seven since the first real system. One production system built, deployed and handed over clean. One local environment running daily on a single machine, where two models and a person share the same memory. A hundred and six architectural decisions and two hundred and eighty-four lessons, written down as they happened, most of them recording something that went wrong.

Three days of human instruction at the start. Everything after that from the systems themselves — and all of it standing on human work.

There is no contradiction between those two sentences, and the space between them is where I have been living.